🎓 **Exam Type: Practical**
⏳ **Duration: 6 Hours**

Natural Language Processing — Review Analytics from Raw Text to Streamlit App

Project Overview

You are a Junior NLP Engineer on the content analytics team of an Indian OTT streaming platform similar to JioCinema or Hotstar. Every day thousands of viewers post text reviews, and the team cannot read them all by hand. Your manager wants a small, working system that cleans raw text, converts it into numbers, learns from it, and tells the team whether a review is positive or negative — wrapped in a simple web app anyone can use.

This is a basic-level exam. You are not expected to reach the highest possible accuracy. You are expected to write clean, working code, explain what each technique does, and show that you understand why you chose it. Steps 1 to 5 build the NLP skills; Step 6 turns your best model into a Streamlit app; Step 7 wraps up with comparison, reflection and GitHub submission.

📋 **Topics covered in this exam:**

Step 1 — NLP Fundamentals, Text Preprocessing & Regular Expressions (RegEx)

Step 2 — Feature Extraction: Bag-of-Words, TF-IDF and N-Grams

Step 3 — Word Embeddings: Word2Vec with CBoW and Skip-gram

Step 4 — Text Classification Pipeline: Sentiment Analysis

Step 5 — Different NLP Approaches (Heuristic, API-style, ML) & POS Tagging

Step 6 — Build the NLP Project and Deploy on Streamlit

Step 7 — Comparison, reflection, and GitHub submission

🎯 Learning Objectives

By completing this exam, students will demonstrate:

- Explaining what NLP is, its common tasks, challenges and the stages of an NLP pipeline.
- Cleaning raw text with RegEx, tokenisation, stop-word removal, stemming and lemmatization.
- Converting text into numeric features using Bag-of-Words, TF-IDF and N-Grams.
- Training Word2Vec models (CBoW and Skip-gram) and exploring word similarity.
- Building, evaluating and comparing text classification pipelines for sentiment analysis.
- Comparing heuristic, ready-made-tool and machine-learning approaches, and using POS tagging.
- Deploying a trained model as a Streamlit web app and submitting reproducible work on GitHub.

🧠 Step 1: NLP Fundamentals, Text Preprocessing & RegEx

📁 **Dataset: SMS Spam Collection**

🔗 UCI: https://archive.ics.uci.edu/dataset/228/sms+spam+collection

🔗 Kaggle mirror: https://www.kaggle.com/datasets/uciml/sms-spam-collection-dataset (file: spam.csv)

📊 5,572 text messages | Columns: v1 (label: ham/spam), v2 (message text)

🎯 Task: Clean raw messages using RegEx and a standard preprocessing pipeline

⚠️ Note: Load with encoding='latin-1'. Keep only the first two columns.

1.1 Theory Notes (Markdown cells)

- What is NLP? Why do businesses need it? Name 3 real applications (for example chatbots at banks, translation of regional languages, review analysis on e-commerce sites).
- List 4 challenges in NLP (for example ambiguity, sarcasm, spelling mistakes, Hinglish code-mixing) with one example sentence each.
- Draw or list the stages of a basic NLP pipeline from raw text to a final prediction.

1.2 Load & Explore

- Load spam.csv with pandas. Rename columns to label and text. Print .shape, .head(5) and the class counts (ham vs spam).
- Plot a countplot of label. What percentage of messages is spam?
- Add a column msg_length (number of characters). Plot the length distribution for spam vs ham on one histogram. Which class has longer messages?

1.3 Regular Expressions (RegEx)

Using Python's re module, write patterns to do the following on the text column:

- Find all messages that contain a URL (pattern such as r'(http|www)\S+'). Report how many are spam and how many are ham.
- Find all messages that contain a phone number of 10 or more digits (pattern such as r'\d{10,}'). Print 3 examples.
- Find all messages that contain a currency symbol (£ or $). Report the spam percentage among them.
- Use re.sub() to replace every number in a message with the token NUM. Show before and after for 3 messages.

💡 Tip: Use df['text'].str.contains(pattern, regex=True) to apply a RegEx to the whole column at once. Test each pattern on a single message first.

1.4 Text Preprocessing Pipeline

Write a function clean_text(text) that applies these steps in order:

- Convert to lowercase.
- Remove URLs, then remove everything that is not a letter or a space using re.sub().
- Tokenise using nltk.word_tokenize().
- Remove English stop words using nltk.corpus.stopwords.
- Apply lemmatization using WordNetLemmatizer.
- Join the tokens back into one string and return it.
- Apply the function to the whole dataset and store the result in a new column clean_text. Show original vs cleaned text for 5 messages.

1.5 Stemming vs Lemmatization

- For these 8 words — running, flies, better, studies, wolves, cars, was, happily — print the Porter stem and the WordNet lemma side by side in a table.
- Write a Markdown cell: what is the difference between stemming and lemmatization? Which one is slower but more accurate, and why?

1.6 Word Frequency

- Find the top 15 most frequent words in spam clean_text and in ham clean_text. Plot two horizontal bar charts.
- Write 2–3 sentences: which words are typical of spam and why would they be useful for a classifier?

⚠️ NLTK downloads needed once: punkt, punkt_tab, stopwords, wordnet, averaged_perceptron_tagger, averaged_perceptron_tagger_eng, vader_lexicon. Run nltk.download('name') for each at the top of your notebook.

🔢 Step 2: Feature Extraction — Bag-of-Words, TF-IDF & N-Grams

📁 **Part A dataset: A tiny 4-sentence corpus (given below)**

Doc 1: the movie was good

Doc 2: the movie was bad

Doc 3: good acting and good story

Doc 4: not a good movie

📁 Part B dataset: The cleaned SMS Spam data from Step 1 (clean_text column)

🎯 Task: Turn text into numbers and understand what each feature type captures

2.1 Bag-of-Words (BoW) on the Tiny Corpus

- Use CountVectorizer() on the 4 documents. Print the vocabulary (get_feature_names_out) and the document-term matrix as a pandas DataFrame with documents as rows.
- Write a Markdown cell: what does each number in the matrix mean? What information does BoW throw away?

2.2 TF-IDF on the Tiny Corpus

- Use TfidfVectorizer() on the same documents. Print the TF-IDF matrix as a DataFrame rounded to 2 decimals.
- Compare the weight of the word movie with the weight of the word acting. Which is higher in the documents where it appears, and why?
- Write a Markdown cell explaining TF, IDF and TF-IDF in your own words with the formula.

2.3 N-Grams

- Run CountVectorizer(ngram_range=(2,2)) and print all bigrams. Do the same with ngram_range=(1,2).
- Find the bigram that comes from Doc 4. Why does a bigram like that help a sentiment model while single words cannot?

2.4 Features on Real Data (SMS Spam)

- Fit CountVectorizer(max_features=3000) on clean_text. Print the matrix shape and the sparsity (percentage of zero entries).
- Fit TfidfVectorizer(max_features=3000) on clean_text. Print the shape. List the 10 words with the highest average TF-IDF score.
- Fit TfidfVectorizer(ngram_range=(1,2), max_features=3000). Print the 10 most frequent bigrams.
- Create a small results table: Method, Number of features, Sparsity %.

💡 Reflection prompt: Why can a machine-learning model not use raw text directly? What do BoW, TF-IDF and N-Grams each add compared with the one before?

🧬 Step 3: Word Embeddings — Word2Vec (CBoW & Skip-gram)

📁 **Dataset: IMDb Dataset of 50K Movie Reviews (used in Steps 3 to 6)**

🔗 Kaggle: https://www.kaggle.com/datasets/lakshmi25npathi/imdb-dataset-of-50k-movie-reviews (file: IMDB Dataset.csv)

📊 50,000 reviews | Columns: review (text), sentiment (positive / negative)

🎯 Task: Train word vectors on movie reviews and explore which words the model thinks are similar

⚠️ Note: Use a balanced sample of 5,000 reviews (2,500 positive + 2,500 negative, random_state=42) to keep training fast.

3.1 Prepare the IMDb Sample

- Load IMDB Dataset.csv. Take the balanced 5,000-review sample. Map sentiment to 1 (positive) and 0 (negative).
- Clean the reviews: remove HTML tags such as
 using re.sub(r'<.*?>', ' ', text), then apply your clean_text logic from Step 1 (you may keep stop words for Word2Vec if you wish — state your choice).
- Create a tokens column (list of words per review). Print the average number of tokens per review.

3.2 Train CBoW and Skip-gram Models

- Train a CBoW model with gensim: Word2Vec(sentences=tokens, vector_size=100, window=5, min_count=5, sg=0, epochs=10, seed=42, workers=1).
- Train a Skip-gram model with the same settings but sg=1.
- Print the vocabulary size and the shape of the vector for the word movie in each model.

3.3 Explore the Embeddings

- For both models, print model.wv.most_similar('good', topn=5) and the same for bad, actor and boring.
- Print the similarity of the pairs (good, great), (good, bad) and (actor, actress) for both models.
- Put the results in a comparison table: Query word, Top-3 similar words in CBoW, Top-3 similar words in Skip-gram.
- Which model gave more sensible neighbours on your data? Write 2–3 sentences.

3.4 Visualise

- Choose 25 words (for example 10 positive words, 10 negative words, 5 film-related words). Get their Skip-gram vectors and reduce to 2D with PCA.
- Make a scatter plot with each point labelled by its word. Do positive and negative words form separate groups?

3.5 Save & Reflect

- Save the Skip-gram model: w2v_model.save('word2vec_skipgram.model').

- Write a Markdown cell: how does CBoW learn (predict the word from its context) versus Skip-gram (predict the context from a word)? Which one handles rare words better? Why is a 100-dimensional dense vector better than a BoW vector for capturing meaning?

💡 With only 5,000 reviews the neighbours will not be perfect — that is expected. Focus on correct training, correct exploration and a clear explanation.

🎯 Step 4: Text Classification Pipeline — Sentiment Analysis

📁 **Dataset: The 5,000-review IMDb sample prepared in Step 3**

🎯 Task: Build a complete text classification pipeline and compare 4 feature methods

📊 Target: sentiment (1 = positive, 0 = negative) | Input: clean review text

4.1 Theory Notes (Markdown cell)

- What is text classification? List the stages of a classification pipeline: raw text → cleaning → features → model → evaluation → prediction.

4.2 Train-Test Split

- Split the cleaned reviews 80% train / 20% test using train_test_split with test_size=0.2, random_state=42, stratify=y.
- Print the sizes and confirm both classes are balanced in train and test.

4.3 Build Four Classifiers

For each method use LogisticRegression(max_iter=1000, random_state=42) as the classifier:

- Model A — Bag-of-Words: Pipeline([CountVectorizer(max_features=5000), LogisticRegression]).
- Model B — TF-IDF: Pipeline([TfidfVectorizer(max_features=5000), LogisticRegression]).
- Model C — TF-IDF with N-Grams: Pipeline([TfidfVectorizer(ngram_range=(1,2), max_features=10000), LogisticRegression]).
- Model D — Averaged Word2Vec: for each review, average the Skip-gram vectors of its tokens (use a zero vector if no token is in the vocabulary). Train LogisticRegression on these 100-dimensional vectors. (This one cannot use a CountVectorizer pipeline — build the feature matrix with a small function.)

4.4 Evaluate & Compare

- For each model print Accuracy, Precision, Recall and F1-Score on the test set.
- Create a comparison table sorted by F1-Score and a bar chart of accuracy for the four models.
- Plot the Confusion Matrix of the best model as a heatmap.
- Write 3–4 sentences: which feature method won, and why do you think so? Did Word2Vec beat TF-IDF? Explain with the size of your training data in mind.

4.5 Inspect the Best Model

- For the best TF-IDF model, print the 10 words with the largest positive coefficients and the 10 with the largest negative coefficients (use the LogisticRegression coef_ and the vectoriser vocabulary).
- Print 5 reviews the model got wrong. For each, write one line on why it may have failed (for example sarcasm, mixed opinion, negation).

4.6 Save the Pipeline

- Save the best pipeline: joblib.dump(best_pipeline, 'sentiment_pipeline.pkl'). It must accept raw text and return a prediction.
- Test the loaded pipeline on 5 new sentences you write yourself. Print sentence, predicted label and probability.

💡 Target accuracy: TF-IDF based models usually reach about 82–88% on this sample; averaged Word2Vec is usually lower (about 70–80%). Do not worry about exact numbers — explain your results.

🔍 Step 5: Different NLP Approaches & POS Tagging

📁 **Dataset: The first 1,000 reviews of your Step 4 test set**

🎯 Task A: Solve the same sentiment problem three different ways and compare

🎯 Task B: Use Part-of-Speech (POS) tagging to see which words describe good and bad movies

5.1 Approach 1 — Heuristic (Rule-Based)

- Write two small word lists yourself: 15 positive words and 15 negative words (for example excellent, boring).
- For each review, score = (number of positive words) − (number of negative words). Predict positive if score ≥ 0, else negative.
- Compute accuracy on the 1,000 test reviews.

5.2 Approach 2 — Ready-Made Tool (API-style)

- Use NLTK's VADER: from nltk.sentiment import SentimentIntensityAnalyzer. Use the compound score — predict positive if compound ≥ 0, else negative.
- Compute accuracy on the same 1,000 reviews. Note: you did not train anything here — you simply called a pre-built tool, exactly like calling a sentiment API.

5.3 Approach 3 — Machine Learning

- Use your saved best pipeline from Step 4 to predict the same 1,000 reviews and compute accuracy.
- Present all three approaches in a table: Approach, Needs training data? (Yes/No), Accuracy, One advantage, One disadvantage.
- Write 3–4 sentences: when would you choose each approach in a real company project?

5.4 POS Tagging

- Write 5 sentences of your own. Use nltk.pos_tag(nltk.word_tokenize(sentence)) and show the result as a table of Word and POS Tag.
- Explain in a Markdown cell the meaning of these tags: NN, NNS, VB, VBD, JJ, RB, DT, IN.
- Take 500 positive and 500 negative reviews (raw text, lowercase only). Extract all words tagged JJ (adjectives). Plot the top 10 adjectives for each class as two bar charts. Which adjectives are most typical of each class?

5.5 How Does POS Tagging Work? (Markdown cell)

- In 4–6 lines explain: why is POS tagging hard (hint: the word book in 'book a ticket' vs 'read a book')? Briefly describe a rule-based tagger and a statistical tagger. Which type is NLTK's default tagger?

💡 Observation to make: the simple rule-based approach is often surprisingly close to the ML model on clear reviews, but fails on negation (for example 'not good at all'). Check whether that happened in your data.

🚀 Step 6: Build the NLP Project and Deploy on Streamlit

📁 **Input: sentiment_pipeline.pkl from Step 4**

🎯 Task: Build a Streamlit web app where a user types a review and instantly sees the predicted sentiment

🔗 Docs: https://docs.streamlit.io | Hosting: https://share.streamlit.io (Streamlit Community Cloud, free)

⚠️ Install with: pip install streamlit. Run with: streamlit run app.py

6.1 Create app.py

- Load the saved pipeline with joblib (wrap the loading in a function decorated with @st.cache_resource).
- The page must contain:
  - A title and a one-line description of the app
  - A text area where the user types or pastes a review
  - An Analyse button
  - The result shown as Positive 😊 or Negative 😞 together with the confidence percentage from predict_proba
  - A sidebar with 3 example reviews the user can read and copy, and a short 'About this project' note
- Handle the empty-input case: if the text area is empty, show st.warning('Please enter a review') instead of a prediction.

6.2 Optional Bonus (counts as extra polish, not extra marks)

- Add an expander that shows the POS tags of the entered review as a small table.
- Add a bar showing the top 3 words that pushed the prediction (using the model coefficients).

6.3 Test Locally

- Run streamlit run app.py. Test with at least 3 inputs: one clearly positive, one clearly negative, one tricky sentence (for example with negation).
- Take screenshots of the app for those 3 cases and save them in the outputs/ folder (for example streamlit_positive.png).

6.4 Deploy

- Push app.py, sentiment_pipeline.pkl and requirements.txt to GitHub.
- Deploy on Streamlit Community Cloud and paste the public app URL in your README.md.
- If the lab network blocks deployment, tell the examiner, run the app locally and demonstrate it in your video. Write 'Deployment blocked — local demo in video' in the README.

⚠️ Version warning: a .pkl file saved with one scikit-learn version may fail on another. Put the exact version (for example scikit-learn==1.5.2) in requirements.txt — use the version printed by your notebook.

📊 Step 7: Comparison, Reflection & GitHub Submission

7.1 Results Summary Table

Create a summary table in a Markdown cell or as a pandas DataFrame covering all six steps:

Step / Topic	RegEx & preprocessing, BoW/TF-IDF/N-Grams, Word2Vec, Classification, Approaches & POS, Streamlit
Dataset used	Name and size of dataset for each step
Key technique	The main library or function used in that step
Key result	Your actual numbers (for example top spam words, vocabulary size, accuracy of each model)
Time taken	Approximate minutes spent on each step

7.2 Written Reflection

Write a Markdown cell (5–8 bullet points) answering:

- Which step was the most challenging and why?
- Which feature method (BoW, TF-IDF, N-Grams, Word2Vec) worked best in your project, and why?
- Give one example sentence where BoW fails but N-Grams succeed.
- What can Word2Vec do that BoW and TF-IDF cannot?
- How did the heuristic approach compare with the ML model, and when is it still a good choice?
- What would go wrong if a user typed a Hinglish review (for example 'picture timepass thi') into your app?
- What would be your next step to improve the project (for example more data, a Hugging Face Transformer, better cleaning)?

7.3 GitHub Submission

- Create a GitHub repository named: nlp-practical-exam-set-a.
- Push the following files:
  - NLP_Practical_Exam_SetA.ipynb (single fully executed notebook for Steps 1–5)
  - sentiment_pipeline.pkl (saved best classification pipeline)
  - word2vec_skipgram.model (plus any .npy files gensim creates alongside it)
  - app.py (Streamlit app)
  - outputs/ folder with plots and Streamlit screenshots
  - summary_report.md (see below)
  - requirements.txt (nltk, scikit-learn, gensim, pandas, numpy, matplotlib, seaborn, streamlit, joblib — with versions)
  - README.md (project overview, steps covered, how to run notebook and app, results table, app URL, video link)

7.4 Summary Report (summary_report.md)

Write ~400–500 words covering:

- What was the business problem and which datasets did you use?

- How did you clean the text and which features did you build?
- Which model and feature method performed best, and what were the numbers?
- How did heuristic, ready-made-tool and ML approaches compare?
- What does your Streamlit app do, and what would you improve next?

Marking Scheme (50 Marks)

Total Marks: 50 — distributed across 3 components as follows:

Component	Marks	Deliverable
A. Technical Completion	30 Marks	Completed .ipynb with all NLP tasks built, run and functional, plus a working Streamlit app
B. Recorded Video (Face + Screen)	15 Marks	5–10 min MP4/MOV showing face + screen with verbal explanation of each NLP technique, results and the app demo
C. GitHub Repository + README.md	5 Marks	Public GitHub repo with saved pipeline, app.py, README, video link, commits
TOTAL	**50 Marks**	

Component B: Recorded Video — Face + Screen (15 Marks)

Record a video showing your face (webcam picture-in-picture) and your computer's entire screen simultaneously. Walk through each step verbally — explain what each technique does, show the outputs, interpret the results and demonstrate your Streamlit app. Do not just click without speaking.

Video Requirements:

Requirement	Detail
Format	MP4 or MOV, maximum 500 MB
Face visibility	Webcam picture-in-picture overlay visible throughout the full recording
Screen content	Entire screen (demonstrating .ipynb, text-processing outputs, model results, and the Streamlit app)
Duration	5 to 10 minutes
Naming convention	Practical_Exam_YourName_GRID.mp4
Upload location	Google Drive (Anyone with link can view) or YouTube (Unlisted)
Link placement	Paste the video URL in your GitHub README.md under the Video section

Component C: GitHub Repository + README.md (5 Marks)

Create a public GitHub repository specifically for this project. Push all files listed in Section 7.3. The README.md must include: project title, list of all steps, dataset links, how to install requirements, how to run the notebook and the app (streamlit run app.py), a results summary table, the deployed app URL (or the local-demo note) and your video link.

📄 Deliverables

Notebook	NLP_Practical_Exam_SetA.ipynb (fully executed, all outputs and plots visible)
Saved Models	sentiment_pipeline.pkl + word2vec_skipgram.model
Streamlit App	app.py + screenshots in outputs/ + deployed URL in README.md
Summary Report	summary_report.md (~1 page, results + reflection)
GitHub Repo	All of the above + requirements.txt + README.md + video link

✅ **Submission Checklist:**

☐ Notebook runs top-to-bottom without errors

☐ Step 1: RegEx patterns, clean_text() function and stemming vs lemmatization table shown

☐ Step 2: BoW, TF-IDF and N-Gram matrices printed for the tiny corpus and the SMS data

☐ Step 3: CBoW and Skip-gram trained, similar-word table and PCA plot shown

☐ Step 4: four classifiers compared in one table, best pipeline saved as .pkl

☐ Step 5: heuristic vs VADER vs ML table and POS-tag adjective charts shown

☐ Step 6: app.py runs with streamlit run app.py, 3 screenshots saved, URL or local-demo note in README

☐ Step 7: results summary table and written reflection complete

☐ GitHub repo link shared with examiner

☐ Video URL pasted in README.md and shared with examiner

BRING ON YOUR CODING ATTITUDE

Practical Exam — Natural Language Processing